Guide de publication Play Store ??? V??rif'All

Guide de publication Play Store — Vérif'All

> Document généré le 29 October 2026 · Référence : `verifall_flutter/docs/publication-play-store/`

Publication sur le Google Play Store — Vérif'All

Ce dossier rassemble ce qu'il faut préparer pour soumettre l'application Android au Play Store. Il ne remplace pas `docs/flutter/07-LIVRAISON.md` du dépôt `verifall_laravel` — qui reste la référence éditoriale sur la signature, les permissions, le versionnement et ce que les deux magasins vont demander — il le **complète** avec des commandes exactes, des valeurs vérifiées dans ce dépôt au 9 octobre 2026, et des documents prêts à remplir ou à coller dans la Console Play.

Principe directeur, identique à celui du reste du projet : on documente ce qui est vrai aujourd'hui, pas ce qui est prévu. Chaque affirmation de ce dossier a été vérifiée dans le code au moment de l'écrire (manifeste, `build.gradle.kts`, `pubspec.yaml`, dépendances) — pas recopiée d'un gabarit générique.

Les fichiers

Fichier	Contenu
`01-SIGNATURE-ET-KEYSTORE.md`	Générer le magasin de clés, activer la signature d'app Play, ce qui est déjà câblé dans `build.gradle.kts`
`02-VERSION-ET-BUILD-AAB.md`	Le numéro de version, comment le faire avancer, la commande exacte pour produire le bundle de publication
`03-PERMISSIONS.md`	Les permissions réellement déclarées aujourd'hui, pourquoi ce ne sont que deux, et ce qui changera le jour où l'audio sera câblé
`04-DATA-SAFETY.md`	De quoi remplir le formulaire « Sécurité des données » de la Console Play, point par point

Fichier	Contenu
`05-POLITIQUE-DE-CONFIDENTIALITE.md`	Brouillon de politique de confidentialité, à faire relire par un juriste avant publication — **ce n'est pas un document juridique validé**
`06-CAPTURES-ECRAN.md`	Les formats exigés par le Play Store, et l'état réel de ce qui peut être produit dans cet environnement aujourd'hui
`07-PUBLICATION.md`	La checklist de soumission, dans l'ordre

Ce qui est déjà prêt, vérifié dans ce dépôt

- `applicationId` : `com.verifall.verifall_app` (`android/app/build.gradle.kts:54`)
- Le mécanisme de signature (`android/key.properties` , non versionné) est câblé et **refuse de construire un bundle de publication sans magasin de clés** — `android/app/build.gradle.kts:116-147` . Il ne manque que le magasin de clés lui-même, qui est un secret d'infrastructure à générer et conserver hors du dépôt (voir `01-SIGNATURE-ET-KEYSTORE.md`).
- `config/prod.json` pointe déjà vers `https://verifall.com` — pas de problème de `localhost` / `10.0.2.2` à résoudre pour un build de publication, cette configuration n'étant utilisée qu'en développement local (`config/exemple.json`).
- Aucun SDK publicitaire, aucun SDK d'analytique tiers (Firebase, Sentry, Facebook, AdMob…) n'est présent dans `pubspec.yaml` — vérifié par recherche textuelle le 9 octobre 2026.
- Permissions déclarées dans le manifeste principal (`android/app/src/main/AndroidManifest.xml`) : **`INTERNET` et `ACCESS_NETWORK_STATE` , et rien d'autre.** Voir `03-PERMISSIONS.md` pour le détail, notamment pourquoi la capture photo/vidéo (déjà codée dans `verify_page.dart`) n'exige pas la permission `CAMERA` .
- Icône de l'application : remplacée le 9 octobre 2026 — l'icône par défaut du gabarit Flutter (le logo bleu « F ») a été remplacée partout (`mipmap-{m,h,xh,xxh,xxxh}dpi/ic_launcher.png` , plus une icône adaptative Android 8+ complète : `ic_launcher_foreground.png` par densité, `values/colors.xml` , `mipmap-anydpi-v26/ic_launcher.xml`) par le vrai logo-icône Vérif'All, tiré de `verifall_laravel/public/assets/images/logo-icon.png` .
- **Les 6 visuels de la fiche (5 captures téléphone + feature graphic) sont livrés** dans `visuels/` — voir `06-CAPTURES-ECRAN.md` pour leur provenance exacte et comment les régénérer.

Ce qui reste à faire, nommé

- **Le magasin de clés n'existe pas encore.** Personne ne peut produire de bundle de publication tant qu'il n'est pas généré — commande exacte dans `01-SIGNATURE-ET-KEYSTORE.md` .
- **La politique de confidentialité est un brouillon**, pas un texte validé — elle doit être relue par quelqu'un d'habilité à engager l'organisation avant d'être publiée à une adresse réelle et déclarée dans la Console Play.
- **Le formulaire « Sécurité des données »** de `04-DATA-SAFETY.md` reflète ce que le code fait réellement aujourd'hui ; il devra être revérifié à chaque fois qu'une nouvelle donnée personnelle est collectée (audio, paiement mobile money, notifications).

Signature et magasin de clés (keystore)

Référence éditoriale : `docs/flutter/07-LIVRAISON.md` §7.1, dépôt `verifall_laravel`. Ce document donne les commandes exactes pour ce dépôt précis.

Ce qui est déjà câblé (rien à écrire)

`android/app/build.gradle.kts` lit `android/key.properties` (non versionné, `.gitignore:61` et `android/.gitignore:12`) et bascule la signature de `release` dessus s'il existe, sinon sur la clé de débogage. Un garde-fou lève une erreur Gradle explicite si quelqu'un lance `flutter build appbundle` **sans** magasin de clés — impossible de publier un bundle signé par accident avec la clé de débogage, qui est la cause la plus fréquente d'application qu'on ne peut plus jamais mettre à jour.

`flutter run --release` continue de fonctionner sans magasin de clés : seul `bundleRelease` (la tâche derrière `flutter build appbundle`) est bloqué.

Le modèle à recopier est `android/key.properties.exemple`.

Étape 1 — Générer le magasin de clés (une seule fois, pour toute la vie de l'app)

```
keytool -genkey -v -keystore ~/verifall-televersement.jks \
  -keyalg RSA -keysize 2048 -validity 10000 -alias televersement
```

`keytool` fait partie du JDK — déjà installé avec Android Studio ou Flutter. Répond aux questions interactives (nom, organisation, pays…) ; elles finissent dans le certificat et ne sont pas secrètes, contrairement au fichier `.jks` lui-même et à ses mots de passe.

Ce fichier `.jks` est un secret d'infrastructure, pas un fichier de projet :

- jamais dans le dépôt Git (il ne l'est pas aujourd'hui — vérifié) ;
- sauvegardé à au moins deux endroits indépendants (ex. gestionnaire de mots de passe de l'équipe + stockage hors ligne) ;
- **le perdre, c'est perdre la capacité de publier une mise à jour** — les personnes qui ont déjà installé l'application ne recevront plus jamais rien, et il n'existe aucune procédure de récupération côté Google pour la clé de distribution si la « signature d'application Play » (étape 2) n'a pas été activée.

Étape 2 — Créer `android/key.properties`

```
cp android/key.properties.exemple android/key.properties
```

Puis remplir :

```
storeFile=/chemin/absolu/vers/verifall-televersement.jks
storePassword=<le mot de passe choisi à l'étape 1>
keyAlias=televersement
keyPassword=<le mot de passe de la clé — souvent le même que storePassword>
```

Ce fichier reste **local à chaque poste qui construit un bundle de publication** — il n'est pas synchronisé entre machines par Git. Sur une CI, les mêmes valeurs arrivent par variables d'environnement/secrets du pipeline, pas par un fichier commité.

Étape 3 — Activer la « signature d'application Play »

C'est la Console Play qui le demande, à la **première** publication :

1. Console Play → votre application → **Production** (ou tout premier canal de test) → **Intégrité de l'application** → **Signature d'application**.
2. Accepter que Google conserve la **clé de distribution**. Le magasin de clés généré à l'étape 1 devient la **clé de téléversement** : c'est elle que chaque nouveau bundle signe, Google la ré-signe avec la clé de distribution avant de la livrer aux téléphones.

C'est le seul filet contre la perte du fichier `.jks` : une clé de téléversement perdue ou compromise se remplace (procédure Google, sur preuve d'identité) ; une clé de distribution perdue ne se remplace jamais. Sans cette étape, le fichier généré à l'étape 1 EST la clé de distribution, et sa perte est définitive.

Étape 4 — Vérifier

```
flutter build appbundle --dart-define-from-file=config/prod.json
```

- **Sans** `android/key.properties` : la construction échoue avec le message du garde-fou (`android/app/build.gradle.kts:130-146`), pas un bundle signé en débogage silencieusement.
- **Avec** : le bundle sort dans `build/app/outputs/bundle/release/app-release.aab`, signé avec la clé de téléversement.

iOS — hors scope de ce document

Vérifié le 9 octobre 2026 : **le dossier `ios/` de ce dépôt n'a pas été configuré pour la distribution** (pas de profil de provisionnement, pas d'identifiants App Store Connect présents dans ce dépôt). La demande d'origine porte sur le Play Store ; la signature iOS (certificats, profils de distribution, capacités déclarées) reste à traiter séparément le jour où la publication App Store est engagée — voir §7.1 de `07-LIVRAISON.md` pour ce que le produit en attend déjà.

Version et construction du bundle (AAB)

Référence éditoriale : `docs/flutter/07-LIVRAISON.md` §7.3, dépôt `verifall_laravel`.

Où vit le numéro de version — une seule source

```
# pubspec.yaml, ligne 9
version: 1.0.0+1
```

`1.0.0` est le **nom de version** (sémantique, visible par les utilisateurs dans le Play Store et dans l'écran Réglages). `1` après le `+` est le **numéro de build** (`versionCode` côté Android) — invisible pour l'utilisateur, mais c'est lui que Play compare pour savoir si un bundle est plus récent.

`android/app/build.gradle.kts:57-60` lit ces deux valeurs via `flutter.versionName` / `flutter.versionCode` — **il n'y a rien à dupliquer ni à synchroniser à la main dans `android/`**. `android/local.properties` contient aujourd'hui `flutter.versionName=1.0.0` et `flutter.versionCode=1` : ce sont des valeurs **mises en cache par**

l'outillage Flutter à partir de `pubspec.yaml`, pas une seconde source de vérité — ce fichier n'est de toute façon pas versionné (`android/.gitignore`) et se régénère tout seul.

L'écran Réglages (30, M5) lit la version depuis le paquet installé via `package_info_plus`, jamais une chaîne écrite en dur — vérifié dans `pubspec.yaml:29` et sa justification en commentaire. Une version qui ment à l'écran Réglages est la première chose qu'on demande à quelqu'un qui signale un défaut ; ce risque est déjà fermé.

Faire avancer la version

Avant **chaque** soumission, même pour un correctif mineur :

```
# Exemple : correctif sans changement visible
# 1.0.0+1 -> 1.0.0+2
#
# Exemple : nouvelle fonctionnalité visible
# 1.0.0+1 -> 1.1.0+2
```

Éditer `pubspec.yaml`, rien d'autre. **Le `versionCode` (après le `+`) doit être strictement croissant et jamais réutilisé** — les deux magasins (Play et App Store) refusent un numéro déjà vu, y compris celui d'un build jamais publié mais déjà téléversé en interne. En cas de doute sur le dernier numéro réellement soumis, vérifier dans la Console Play → Production → Historique des versions plutôt que de deviner.

Construire le bundle de publication

```
flutter build appbundle --dart-define-from-file=config/prod.json
```

- Exige `android/key.properties` (voir `01-SIGNATURE-ET-KEYSTORE.md`) — échoue sinon, volontairement.
- `--dart-define-from-file=config/prod.json` est **obligatoire** : sans lui, `AppConfig.apiBaseUrl` est une chaîne vide, et `assertNoDeveloperSessionInProduction()` (`lib/core/config/app_config.dart`) refuse de démarrer l'application en environnement `production` sans adresse d'API — la construction réussirait, l'app s'installerait, et chaque appel échouerait sans qu'aucun message n'explique pourquoi, exactement le défaut que ce garde-fou existe pour fermer.
- Sortie : `build/app/outputs/bundle/release/app-release.aab`.

Ne pas confondre avec l'APK

```
flutter build apk --dart-define-from-file=config/prod.json
```

Produit un `.apk`, pas un `.aab`. Utile pour un test manuel sur un appareil physique sans passer par Play, **mais le Play Store exige le format App Bundle** pour toute soumission — voir `07-LIVRAISON.md` §7.1. La vérification `bundleRelease` du garde-fou de signature ne se déclenche que sur la tâche de bundle, précisément pour ne pas bloquer ce cas d'usage local.

Avant de construire pour de vrai

```
flutter analyze          # 0 issue
flutter test              # tous les tests passent
dart format . --set-exit-if-changed   # aucun fichier reformate
```

Ces trois commandes sont déjà la barre de qualité du projet (voir le README racine) — les relancer juste avant une construction de publication coûte quelques minutes et évite de découvrir un défaut après la revue Google, qui prend plusieurs jours à chaque aller-retour.

Permissions Android

Référence éditoriale : `docs/flutter/07-LIVRAISON.md` §7.2, dépôt `verifall_laravel` (« toute permission supplémentaire est un refus d'installation en puissance — et, sur un produit de fact-checking, un soupçon »). Ce document vérifie l'état RÉEL, pas l'état visé.

Ce qui est déclaré aujourd'hui — et rien de plus

`android/app/src/main/AndroidManifest.xml` (manifeste de release, celui qui compte pour le Play Store) :

```
<uses-permission android:name="android.permission.INTERNET"/>
<uses-permission android:name="android.permission.ACCESS_NETWORK_STATE"/>
```

- **`INTERNET`** : obligatoire, posée explicitement dans le manifeste principal (pas seulement `debug` / `profile` où Flutter la pose par défaut) — toute la donnée de l'app vient de l'API. Le commentaire en place dans le manifeste explique pourquoi cette ligne ne peut pas être oubliée : sans elle, un APK de release se lancerait normalement et échouerait sur **chaque** appel réseau, un mode de panne invisible en développement.
- **`ACCESS_NETWORK_STATE`** : pour `connectivity_plus`. Sert d'indice d'état réseau à l'interface, jamais de verrou — l'app n'empêche pas une tentative sans connexion, elle prévient que ça risque d'échouer.

Vérifié aussi : **aucun greffon du projet ne déclare de permission supplémentaire dans son propre manifeste** — contrôlé le 9 octobre 2026 sur `image_picker_android`, `video_player_android`, `share_plus`, `url_launcher_android`, `device_info_plus`, `flutter_secure_storage`, `connectivity_plus`. Le manifeste final (après fusion Gradle) ne contient donc que les deux lignes ci-dessus.

Le cas qui surprend : la caméra n'a pas besoin de permission `CAMERA`

`lib/presentation/features/verify/pages/verify_page.dart` appelle déjà `ImagePicker().pickImage(source: ImageSource.camera)` et `pickVideo(...)` — la capture photo/vidéo **est** codée, contrairement à ce qu'on pourrait supposer en lisant seulement §7.2 de `07-LIVRAISON.md`, qui range cet usage sous « écran 09, pas encore câblé ».

Et pourtant, aucune permission `CAMERA` n'apparaît nulle part. Ce n'est pas un oubli : `image_picker` sur Android délègue la capture à l'application Appareil photo du système par un **intent implicite** (`MediaStore.ACTION_IMAGE_CAPTURE` / `ACTION_VIDEO_CAPTURE`) — c'est l'application Caméra qui détient la permission, jamais l'appelant. C'est exactement pourquoi le manifeste du plugin ne déclare rien : vérifié directement dans son code, pas supposé.

Conséquence pratique : ne pas ajouter `android.permission.CAMERA` à la main. L'ajouter sans l'utiliser directement (ex. via le greffon natif Android `Camera2`) ferait échouer la revue Google sur la règle citée plus haut — une permission déclarée doit être justifiable par un usage réel dans l'app.

Ce qui N'EST PAS déclaré, et pourquoi c'est correct aujourd'hui

Permission	Écran qui la justifierait (§7.2)	État du code au 9 octobre 2026
`RECORD_AUDIO` (microphone)	Enregistrer une note vocale à vérifier	**Non câblé.** `verify_page.dart` affiche une boîte de dialogue « Import audio indisponible » dès que `TypeContenu.audio` est choisi — aucun code n'accède au microphone.
`POST_NOTIFICATIONS`	Prévenir qu'un verdict est prêt	**Non câblé.** `VF_FEATURE_NOTIFICATIONS=false` dans toutes les configurations (`config/*.json`) ; aucun appel à un plugin de notifications dans `pubspec.yaml` .

Chacune de ces deux lignes redevient à écrire le jour où l'écran correspondant est livré — ni avant (une permission inutilisée se refuse), ni après coup dans la précipitation (une demande de permission ajoutée après la première publication doit être justifiée dans la fiche du magasin au prochain envoi).

Ce qui ne doit jamais être ajouté

Repris de `07-LIVRAISON.md` §7.2, toujours vrai aujourd'hui : **localisation** (arbitrage n° 9, refus explicite de géolocaliser les sessions), **contacts**, **stockage externe**, **calendrier**. Aucun de ces quatre n'a de justification dans le produit tel qu'il est écrit.

La règle qui ne se vérifie pas dans un manifeste

« Les permissions sont demandées **au moment de l'usage**, jamais au lancement » (§7.2) est un comportement d'exécution, pas une déclaration XML — rien dans ce document ne le garantit statiquement. À revérifier manuellement à l'écran au moment où une permission runtime sera effectivement demandée (ce qui n'est pas encore le cas aujourd'hui, caméra comprise, puisqu'elle ne demande rien directement).

Formulaire « Sécurité des données » (Google Play Console)

Référence éditoriale : `docs/flutter/07-LIVRAISON.md` §7.5 et §7.6, dépôt `verifall_laravel` (« sous-déclarer est un motif de retrait »). Ce document traduit ce que le code fait réellement, au 9 octobre 2026, dans le vocabulaire exact des catégories du formulaire Play Console (section **App content → Data safety**). Il ne remplace pas le jugement de la personne qui le remplit dans la Console — c'est un relevé de faits, pas le formulaire rempli.

Méthode

Pour chaque catégorie Play, trois questions : **la donnée est-elle collectée ? Est-elle partagée avec un tiers ? Est-elle optionnelle ?** — et la preuve dans le code.

Informations personnelles

Donnée	Collectée ?	Preuve dans le code	Partagée avec un tiers ?
Adresse e-mail	Oui	`auth_remote_datasource.dart` — connexion, inscription, réinitialisation de mot de passe	Non — reste sur l'API Vérif'All
Nom	Non constaté dans ce dépôt au-delà de ce que `lib/` échange avec `/auth/*`	—	—
Téléphone	Non constaté	—	—

Finalité à déclarer : fonctionnalité de l'application (authentification du compte). **Suppression** : l'écran `/compte-en-suppression` existe (`lib/core/router/routes_profile.dart:123` , `vf_routes.dart:82`) — la suppression de compte est bien **accessible depuis l'application**, exigence explicite des deux magasins (§7.5).

Contenu utilisateur

Donnée	Collectée ?	Preuve	Note
Texte soumis à vérification (l'affirmation, le contexte)	Oui	Flux de soumission, `verify_page.dart` → API	C'est **le cœur du produit** ; `07-LIVRAISON.md` §7.6 l'exclut explicitement de toute télémétrie — seule l'API de vérification le reçoit, jamais un fournisseur de crash analytics
Photos	Oui, si l'utilisateur en soumet une	`ImagePicker().pickImage` dans `verify_page.dart`	Transmise au serveur pour analyse, pas conservée ailleurs côté client au-delà du cache de soumission hors ligne
Vidéos	Oui, si l'utilisateur en soumet une	`ImagePicker().pickVideo` , `video_player` / `chewie` pour la relecture	Idem
Audio	**Non** — fonctionnalité non câblée	`verify_page.dart` affiche « Import audio indisponible »	Rien à déclarer tant que ce n'est pas livré ; à revérifier à ce moment-là

Identifiants

Donnée	Collectée ?	Preuve	Finalité
Jeton d'authentification	Oui, stocké	`lib/core/network/token_store.dart` , via `flutter_secure_storage` (coffre chiffré du système,	Fonctionnalité de l'app — rester

Donnée	Collectée ?	Preuve	Finalité
		jamais `SharedPreferences`)	connecté
Modèle de l'appareil	Oui, envoyé au serveur	`lib/data/datasources/remote/auth/nom_appareil.dart`, via `device_info_plus`	Associé à l'émission d'un jeton (traçabilité des sessions actives, probablement affichée en Réglages) — **à confirmer côté produit avant de remplir le formulaire : vérifier si l'écran Réglages affiche une liste d'appareils connectés**
Identifiant d'installation anonyme	Généré, **mais pas transmis aujourd'hui**	`lib/core/telemetry/identifiant_installation.dart`	Pensé pour regrouper des incidents de plantage — `telemetryDsn` est vide dans toutes les configurations (`config/*.json`) et aucun fournisseur n'est branché (`PuitsDeTelemetrie` n'a aucune implémentation active, §7.6 du document de livraison). **Ne rien déclarer de transmis tant qu'aucun fournisseur n'est effectivement branché** ; ce tableau devra être corrigé le jour où `main.dart` en configure un

Ce qui n'est PAS collecté — vérifié, pas supposé

- **Localisation** : aucun package de géolocalisation dans `pubspec.yaml`, et l'arbitrage n° 9 du projet la refuse explicitement pour les sessions.
- **Contacts, calendrier, stockage externe** : aucun package correspondant.
- **Publicité / identifiants publicitaires** : aucun SDK publicitaire.
- **Analytique tierce** (Firebase Analytics, Facebook SDK, etc.) : absente de `pubspec.yaml`, vérifié le 9 octobre 2026.
- **Adresse IP en clair** : le back impose `IdentityHasher` côté serveur (§7.6) — rien à déclarer côté client, mais cette ligne concerne le dépôt `verifall_laravel`, pas celui-ci ; à confirmer que la pratique serveur

tient toujours au moment de remplir le formulaire.

Chiffrement en transit

`AppConfig.assertNoDeveloperSessionInProduction()` (`lib/core/config/app_config.dart`) **refuse de démarrer l'application en production si `apiBaseUrl` ne commence pas par `https://`** — ce n'est pas une politique déclarée, c'est une assertion exécutée au démarrage. Répondre « oui » à « les données sont-elles chiffrées en transit » est donc vérifiable dans le code, pas une simple déclaration d'intention.

Point à trancher avant de remplir le formulaire

Paiement / mobile money. `docs/flutter/07-LIVRAISON.md` et le reste de la documentation du produit mentionnent un système d'unités et de recharge, mais **aucun flux de paiement n'a été trouvé dans `lib/` de ce dépôt** au 9 octobre 2026 (aucune dépendance `in_app_purchase`, aucun datasource mobile money côté Flutter). Si la recharge se fait aujourd'hui uniquement par le site web (`verifall.com`), le formulaire de l'app n'a rien à déclarer sur ce point ; si elle est prévue côté app avant la soumission, ce tableau devra être complété avec les champs réellement envoyés (numéro de téléphone opérateur, montant…) et une catégorie « Informations financières » s'ajoutera au formulaire. **Cette décision n'est pas technique — elle revient à qui pilote le périmètre de la première version publiée.**

Politique de confidentialité

Elle existe déjà — ne pas en écrire une seconde

Vérifié le 9 octobre 2026 : `verifall_laravel/resources/views/public/confidentialite.blade.php` publie une politique de confidentialité complète et structurée (responsable du traitement, bases légales, données collectées, destinataires et sous-traitants, durées de conservation, droits RGPD, mineurs), servie en production à **`https://verifall.com/confidentialite`** — confirmé joignable (HTTP 200) à cette même date.

Ne pas rédiger un second texte pour l'application mobile. Une politique de confidentialité dupliquée est un risque, pas une précaution : les deux versions divergent tôt ou tard, et le jour où elles le font, aucune des deux n'est plus fiable. L'application a déjà le mécanisme pour pointer vers l'unique version qui fait foi — voir ci-dessous.

Comment l'application mobile s'y relie déjà

`lib/core/config/app_config.dart` porte `webBaseUrl` (`VF_WEB_BASE_URL` dans `config/*.json`, déjà renseignée à `https://verifall.com` dans `config/prod.json`). Le commentaire du champ dans le code le dit explicitement : *« M5 a retiré du menu la ligne "Politique de confidentialité" plutôt que de la laisser inerte… Renseignée, les deux deviennent des liens. »* (arbitrage n° 138 cité dans le fichier).

Conséquence pratique : une fois le bundle construit avec `config/prod.json` (voir `02-VERSION-ET-BUILD-AAB.md`), l'écran Réglages affiche déjà un lien cliquable vers `https://verifall.com/confidentialite`. Il n'y a rien à coder pour ce document précis — seulement à vérifier que le lien part bien vers la bonne page avant la première soumission.

Ce qu'il reste à faire, et qui n'est pas du code

1. **Renseigner le champ « URL de la politique de confidentialité » dans la Google Play Console** (Contenu de l'application → Politique de confidentialité) avec `https://verifall.com/confidentialite`.
2. **Faire relire le texte publié** par quelqu'un d'habilité à engager l'organisation, en vérifiant qu'il couvre bien ce que l'**application mobile** fait spécifiquement et que la page web ne mentionne peut-être pas explicitement :
   - le modèle de l'appareil envoyé au serveur à l'émission d'un jeton (`lib/data/datasources/remote/auth/nom_appareil.dart`) ;
   - le stockage local chiffré du jeton d'authentification (`flutter_secure_storage`) ;
   - l'identifiant d'installation anonyme généré pour la télémétrie de plantage (`lib/core/telemetry/identifiant_installation.dart`) — actuellement généré mais **non transmis** (voir `04-DATA-SAFETY.md`), donc rien à déclarer tant qu'aucun fournisseur n'est branché, mais à ajouter au texte le jour où `main.dart` en configure un.

   La page actuelle couvre déjà, en des termes génériques, les « données techniques » et les « contenus que vous soumettez » — à confirmer que cette formulation suffit pour les trois points mobiles ci-dessus, ou si une section dédiée à l'application devient nécessaire.
3. Vérifier que les réglages dynamiques dont dépend la page (`legal_entity`, `legal_address`, `legal_host_name`, `legal_contact_donnees` — lus via `$legal->entite()`, etc. dans le gabarit Blade) sont **remplis** en production. Un responsable du traitement non renseigné affiche un champ vide ou un repli, ce qui est un motif de rejet aussi sûr qu'une politique absente.

Ce que ce document ne fait pas

Il ne reproduit pas le texte de la politique — elle vit à un seul endroit, `confidentialite.blade.php`, et toute modification se fait là, jamais ici.

Captures d'écran et visuels de la fiche Play Store

Livré — 6 fichiers dans `visuels/`

Fichier	Dimensions	Format	Poids
`screenshot-1-accueil.png`	1080 × 1920	PNG 24 bits, sans alpha	~0,9 Mo — **montage** 3×2 (Vérification, Académie, Quiz, Certificat, Forum, Opportunités), pas le carrousel de connexion
`screenshot-2-tableau-de-bord.png`	1080 × 1920	idem	~0,9 Mo
`screenshot-3-verifier.png`	1080 × 1920	idem	~0,8 Mo
`screenshot-4-rapport.png`	1080 × 1920	idem	~0,9 Mo

Fichier	Dimensions	Format	Poids
`screenshot-5-academie.png`	1080 × 1920	idem	~0,9 Mo
`feature-graphic.png`	1024 × 500	idem	~0,3 Mo

Vérifié : chaque fichier est un vrai PNG 24 bits **sans** canal alpha (`System.Drawing.Image.PixelFormat` = `Format24bppRgb`), sous la limite de 8 Mo par image exigée par le magasin.

D'où vient le contenu des téléphones

Pas du binaire Flutter. Deux voies ont été tentées pour capturer l'app réellement construite, et les deux étaient bloquées dans cet environnement :

- **Émulateur Android** : le téléchargement de l'image système échouait systématiquement (connexion coupée par le logiciel hôte, probablement un antivirus/pare-feu local) ;
- **Flutter Web** : `flutter_html 3.0.0` (utilisé par `VfProseBody` pour le rendu des leçons et du rapport) ne compile pas pour le web — `Method not found: 'matches'` dans son propre code, un bug amont indépendant de ce projet.

La source réelle est `prototype_html/mobile/index.html`, le prototype interactif à 30 écrans qui fait déjà foi pour la transposition Flutter (« on transpose, on n'invente pas », README du dépôt Flutter). Chaque écran a été ouvert via son lien profond (`#/dashboard` , `#/verify` , `#/report` , `#/academy` , `#/quiz` , `#/certificate` , `#/forum` , `#/hub` , `#/` pour le splash), capturé avec le binaire `chrome-headless-shell` (installé via le skill `hyperframes` , indépendant de l'extension Chrome qui s'est montrée instable dans cette session), puis détouré à la silhouette exacte du châssis de téléphone déjà dessiné par le prototype (coins arrondis, encoche, boutons).

`screenshot-1-accueil.png` capture le carrousel d'accueil de façon déterministe via `chrome-headless-shell --virtual-time-budget=N` : le carrousel défile tout seul (4,2 s/diapositive), donc une capture sans ce réglage tombe sur une diapositive aléatoire selon la vitesse de chargement — `budget=0` pour la diapositive 1, `12600` + pour la 4e. C'est ce qui a révélé, lors d'une capture précédente tombée par hasard sur la diapositive 4, que son texte avait besoin d'être retravaillé (voir plus bas).

Ce qui a été corrigé dans le prototype au passage, à la demande explicite de l'utilisateur : - le vrai logo et le vrai favicon de Vérif'All ont remplacé le glyphe texte « V » et l'absence de favicon (`<link rel="icon">` ajouté, image embarquée en base64 depuis `verifall_laravel/public/assets/images/logo-icon.png` et `favicon.png`) ; - la diapositive 04 du carrousel d'accueil a été réécrite le 9 octobre 2026 : le texte d'origine (« Un communiqué officiel truffé de fautes… ») répétait son propre titre (« La forme trahit souvent le fond » en double) et tombait, selon le moment de capture, sur un bloc trop dense. Nouveau texte, même thème (la forme comme indice), même gabarit que les diapositives 1 à 3 — vérifié sans débordement via une capture déterministe (`--virtual-time-budget=13000`). La fonction de rendu a aussi été corrigée pour ne plus afficher de guillemets vides quand les champs sont absents (robustesse, indépendante du contenu).

Ce qui habille chaque capture

Gabarit HTML/CSS propre (pas les skills « hyperframes »/« ui-ux-pro-max-skill-main » de la demande d'origine — ni l'une ni l'autre ne produit directement un visuel de fiche Play Store ; la composition a été faite à la main avec les jetons de couleur réels du design system Flutter, `VfColors.forest` / `primary`) : fond dégradé vert forêt de marque, pastille « VÉRIF'ALL », titre et sous-titre sans tiret ni tiret cadratin/demi-cadratin

(contrainte explicite), logo réel en haut à gauche, téléphone capturé au centre, signature « La boussole contre la désinformation numérique » en pied de page.

Reproduire ou régénérer

Les fichiers de travail (gabarits HTML, captures brutes, scripts de recadrage PowerShell) ne sont pas versionnés dans ce dépôt — ils ont vécu dans un répertoire de travail temporaire le temps de la session. Pour regénérer :

1. Servir `prototype_html/mobile/index.html` en local (`python -m http.server`).
2. Capturer chaque écran avec `chrome-headless-shell --headless --screenshot=out.png --window-size=1400,1150 "http://localhost:PORT/index.html#/ROUTE"` pour `ROUTE` ∈ `{(vide)=splash, dashboard, verify, report, academy}` .
3. Recadrer la zone du châssis (`392,235` à `+438,+880` dans cette mise en page) avec un masque de coins arrondis (rayon 58 px) pour obtenir un détourage propre.
4. Composer dans un gabarit 1080 × 1920 (téléphone) ou 1024 × 500 (feature graphic) avec le fond de marque et le texte.

Captures tablette / Chromebook

Non produites. L'application ne déclare aucun support tablette ou Chromebook explicite à ce jour (`03-PERMISSIONS.md` , aucune configuration de form factor trouvée) — à constituer séparément le jour où ce périmètre est confirmé.

Ce qui reste vrai de l'avertissement d'origine

Le contenu affiché (verdicts, cours, KPI) vient du prototype de référence, pas d'un compte réel ni de données de production en direct — c'est la même donnée de démonstration que celle que l'équipe Flutter utilise déjà pour valider chaque écran contre ce prototype. Avant la soumission définitive, vérifier que rien ne ressemble à un engagement produit non tenu (ex. chiffres de démonstration trop spécifiques) plutôt qu'à une illustration.

Checklist de soumission — dans l'ordre

Chaque étape renvoie au document qui la détaille. Cocher dans l'ordre : les étapes de construction dépendent de celles de signature, qui doivent être faites une seule fois et pour toujours.

1. Une seule fois, pour toute la vie de l'application

- ☐ Générer le magasin de clés (`01-SIGNATURE-ET-KEYSTORE.md` , étape 1).
- ☐ Créer `android/key.properties` à partir de `key.properties.exemple` (étape 2) — **ne jamais le committer**.
- ☐ Première soumission seulement : activer la **signature d'application Play** dans la Console (étape 3). Sans elle, le fichier `.jks` généré à l'étape précédente devient lui-même la clé de distribution, et sa perte devient définitive.
- ☐ Déclarer `https://verifall.com/confidentialite` comme politique de confidentialité dans la Console Play (`05-POLITIQUE-DE-CONFIDENTIALITE.md`).
- ☐ Faire relire cette politique par quelqu'un d'habilité à engager l'organisation, en particulier sur les trois points propres au mobile listés dans ce même document.

2. À chaque soumission

- [ ] Faire avancer `version:` dans `pubspec.yaml` (`02-VERSION-ET-BUILD-AAB.md`) — le numéro après `+` ne doit jamais avoir déjà été soumis, pas même en interne.
- [ ] `flutter analyze` → 0 issue.
- [ ] `flutter test` → tout passe.
- [ ] `dart format . --set-exit-if-changed` → aucun fichier reformaté.
- [ ] Construire : `flutter build appbundle --dart-define-from-file=config/prod.json`.
- [ ] Revérifier `03-PERMISSIONS.md` : une permission a-t-elle été ajoutée depuis la dernière soumission ? Si oui, son usage doit être visible et justifiable dans la fiche du magasin.
- [ ] Revérifier `04-DATA-SAFETY.md` : une nouvelle donnée personnelle est-elle collectée depuis la dernière soumission (audio, paiement, notifications) ? Si oui, mettre à jour le formulaire **avant** de soumettre — le corriger après coup, une fois le rejet reçu, coûte des jours.
- [ ] Captures d'écran à jour si l'interface a changé de façon visible depuis les 6 visuels livrés le 9 octobre 2026 dans `visuels/` (`06-CAPTURES-ECRAN.md`).
- [ ] Note de version **en français, qui dit ce qui change pour la personne**, pas ce qui change dans le code (§7.3 du document de livraison).

3. Diffusion — trois canaux, dans cet ordre (§7.4)

1. **Interne** — l'équipe, à chaque compilation de la branche principale.
2. **Recette / test fermé** — groupe restreint, les cinq parcours du produit validés avant d'aller plus loin.
3. **Production, déploiement progressif** — commencer à une petite fraction des utilisateurs, suivre le taux de plantage, élargir ensuite. Une fois à 100 %, un correctif de régression attend la prochaine revue — c'est le coût d'avoir sauté l'étape de progressivité, pas une option de confort.

4. Après publication

- [ ] Consigner ce qui est livré, ce qui ne l'est pas, et les écarts au simulateur dans `docs/ARBITRAGES.md` du dépôt — « dire ce qui n'est pas fait, un rapport qui tait une réserve est pire qu'un retard annoncé » (§7.7).
- [ ] Suivre le taux de plantage réel reste impossible tant qu'aucun fournisseur de télémétrie n'est branché (voir `04-DATA-SAFETY.md`, §7.6 du document de livraison) — **ce n'est pas un oubli de cette checklist, c'est une décision produit non tranchée** qui mérite d'être reprise avant une diffusion à grande échelle : sans elle, un plantage massif en production ne se voit que par les avis une étoile.